241801053- J B Dhejasvi Dynamic Programming-Knapsack problem

1. Basic 0–1 Knapsack Problem A knapsack can carry a maximum weight of 15 kg. Four items are available: Item 1: Profit = 10, Weight = 2 Item 2: Profit = 12, Weight = 4 Item 3: Profit = 18, Weight = 6 Item 4: Profit = 25, Weight = 9 Problem: Formulate the 0–1 knapsack model and using dynamic programming, determine which items should be selected to maximize total profit.

```
# 0-1 Knapsack using Dynamic Programming

profits = [10, 12, 18, 25]
weights = [2, 4, 6, 9]
capacity = 15
n = len(profits)

# DP table
dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

# Build table
for i in range(1, n + 1):
    for w in range(1, capacity + 1):
        if weights[i-1] <= w:
            dp[i][w] = max(profits[i-1] + dp[i-1][w-weights[i-1]],
                           dp[i-1][w])
        else:
            dp[i][w] = dp[i-1][w]

# Maximum profit
max_profit = dp[n][capacity]

# Find selected items
w = capacity
selected_items = []

for i in range(n, 0, -1):
    if dp[i][w] != dp[i-1][w]:
        selected_items.append(i)
        w -= weights[i-1]

selected_items.reverse()

# Calculate total weight
total_weight = sum(weights[i-1] for i in selected_items)

print("Selected Items:", selected_items)
print("Total Weight:", total_weight)
print("Maximum Profit:", max_profit)
```

```
Selected Items: [1, 2, 4]
Total Weight: 15
Maximum Profit: 47
```

2. 0–1 Knapsack Problem (Medium) A knapsack has a capacity of 20 units. The following items are available: Item 1: Profit = 24, Weight = 6 Item 2: Profit = 13, Weight = 4 Item 3: Profit = 23, Weight = 5 Item 4: Profit = 15, Weight = 3 Item 5: Profit = 16, Weight = 7 Problem: Formulate the 0–1 knapsack model and using dynamic programming, determine which items should be selected to maximize total profit.

```
# 0-1 Knapsack using Dynamic Programming

profits = [24, 13, 23, 15, 16]
weights = [6, 4, 5, 3, 7]
capacity = 20
n = len(profits)

# DP table
dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

# Fill DP table
for i in range(1, n + 1):
    for w in range(1, capacity + 1):
        if weights[i-1] <= w:
            dp[i][w] = max(profits[i-1] + dp[i-1][w - weights[i-1]],
```

```
                            dp[i-1][w])
        else:
            dp[i][w] = dp[i-1][w]

# Maximum profit
max_profit = dp[n][capacity]

# Find selected items
w = capacity
selected_items = []

for i in range(n, 0, -1):
    if dp[i][w] != dp[i-1][w]:
        selected_items.append(i)
        w -= weights[i-1]

selected_items.reverse()

# Total weight
total_weight = sum(weights[i-1] for i in selected_items)

print("Selected Items:", selected_items)
print("Total Weight:", total_weight)
print("Maximum Profit:", max_profit)
```

```
Selected Items: [1, 2, 3, 4]
Total Weight: 18
Maximum Profit: 75
```

3. Investment Selection: A company wants to invest in several projects. The available capital is 14 units. Project A: Profit = 20, Investment = 4 Project B: Profit = 15, Investment = 3 Project C: Profit = 25, Investment = 6 Project D: Profit = 18, Investment = 5 Tasks:
4. Formulate this as a 0–1 knapsack model.
5. Determine the best combination of project

```
# Investment Selection using 0-1 Knapsack

profits = [20, 15, 25, 18]
investments = [4, 3, 6, 5]
capacity = 14
n = len(profits)

# DP table
dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

# Build table
for i in range(1, n + 1):
    for w in range(1, capacity + 1):
        if investments[i-1] <= w:
            dp[i][w] = max(profits[i-1] + dp[i-1][w - investments[i-1]],
                           dp[i-1][w])
        else:
            dp[i][w] = dp[i-1][w]

# Maximum profit
max_profit = dp[n][capacity]

# Find selected projects
w = capacity
selected_projects = []

for i in range(n, 0, -1):
    if dp[i][w] != dp[i-1][w]:
        selected_projects.append(i)
        w -= investments[i-1]

selected_projects.reverse()

# Total investment
total_investment = sum(investments[i-1] for i in selected_projects)

print("Selected Projects:", selected_projects)
print("Total Investment:", total_investment)
print("Maximum Profit:", max_profit)
```

```
Selected Projects: [1, 2, 3]
Total Investment: 13
Maximum Profit: 60
```

4. Cargo Loading Problem: A small cargo plane can carry 12 tons of weight. The following cargo packages are available: Cargo 1: Profit = 8, Weight = 2 Cargo 2: Profit = 10, Weight = 4 Cargo 3: Profit = 15, Weight = 6 Cargo 4: Profit = 4, Weight = 1 Tasks:
5. Formulate the 0–1 knapsack mathematical model.
6. Find the optimal loading plan.

```
# Cargo Loading using 0-1 Knapsack

profits = [8, 10, 15, 4]
weights = [2, 4, 6, 1]
capacity = 12
n = len(profits)

# DP table
dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

# Fill DP table
for i in range(1, n + 1):
    for w in range(1, capacity + 1):
        if weights[i-1] <= w:
            dp[i][w] = max(profits[i-1] + dp[i-1][w-weights[i-1]],
                           dp[i-1][w])
        else:
            dp[i][w] = dp[i-1][w]

# Maximum profit
max_profit = dp[n][capacity]

# Find selected cargo
w = capacity
selected = []

for i in range(n, 0, -1):
    if dp[i][w] != dp[i-1][w]:
        selected.append(i)
        w -= weights[i-1]

selected.reverse()

total_weight = sum(weights[i-1] for i in selected)

print("Selected Cargo:", selected)
print("Total Weight:", total_weight)
print("Maximum Profit:", max_profit)
```

```
Selected Cargo: [1, 2, 3]
Total Weight: 12
Maximum Profit: 33
```

5. Large Knapsack Problem Capacity of knapsack = 10 units Item 1: Profit = 5, Weight = 2 Item 2: Profit = 8, Weight = 3 Item 3: Profit = 14, Weight = 5 Item 4: Profit = 6, Weight = 4 Item 5: Profit = 7, Weight = 3 Task: Find the optimal selection of items.

```
# 0-1 Knapsack Problem

profits = [5, 8, 14, 6, 7]
weights = [2, 3, 5, 4, 3]
capacity = 10

n = len(profits)

# DP table
dp = [[0]*(capacity+1) for _ in range(n+1)]

# Fill table
for i in range(1, n+1):
    for w in range(1, capacity+1):
        if weights[i-1] <= w:
            dp[i][w] = max(profits[i-1] + dp[i-1][w-weights[i-1]],
                           dp[i-1][w])
        else:
            dp[i][w] = dp[i-1][w]
```

```
            dp[i][w] = dp[i-1][w]

max_profit = dp[n][capacity]

# Backtrack to find items
w = capacity
items = []

for i in range(n, 0, -1):
    if dp[i][w] != dp[i-1][w]:
        items.append(i)
        w -= weights[i-1]

items.reverse()

total_weight = sum(weights[i-1] for i in items)

print("Selected Items:", items)
print("Total Weight:", total_weight)
print("Maximum Profit:", max_profit)
```

```
Selected Items: [1, 2, 3]
Total Weight: 10
Maximum Profit: 27
```

6. A bag has capacity 8 kg. Item 1: Profit = 6, Weight = 2 Item 2: Profit = 10, Weight = 3 Item 3: Profit = 12, Weight = 4 Item 4: Profit = 7, Weight = 3

```
# 0-1 Knapsack Problem

profits = [6, 10, 12, 7]
weights = [2, 3, 4, 3]
capacity = 8

n = len(profits)

# DP table
dp = [[0]*(capacity+1) for _ in range(n+1)]

for i in range(1, n+1):
    for w in range(1, capacity+1):
        if weights[i-1] <= w:
            dp[i][w] = max(profits[i-1] + dp[i-1][w-weights[i-1]],
                           dp[i-1][w])
        else:
            dp[i][w] = dp[i-1][w]

max_profit = dp[n][capacity]

# Backtracking to find items
w = capacity
items = []

for i in range(n,0,-1):
    if dp[i][w] != dp[i-1][w]:
        items.append(i)
        w -= weights[i-1]

items.reverse()

total_weight = sum(weights[i-1] for i in items)

print("Selected Items:", items)
print("Total Weight:", total_weight)
print("Maximum Profit:", max_profit)
```

```
Selected Items: [1, 2, 4]
Total Weight: 8
Maximum Profit: 23
```